Temco Platform

`temco-projects` · built Nov 10, 2026

`temco-projects`

A documentation project as a first-class thing: its name, its sources, its outputs, its builds, and the pages it produces. One page — the edit form — and one command that opens the project's pages.

Folder	`extensions/temco-projects`
Extension id	`temcocontrols.temco-projects`
Namespace	`temcoProjects`
State	active
Pages	`temcoProjects.project` (the form) — opening a project shows its pages through `temcoDocs.documents`
Settings	none
Requires	`temco-store` (memory), `temco-ui` (page columns), `temco-repositories` (sources and their scripts)
Required by	`temco-docs` — for the one signal that brings the Library's project rows back

Responsibilities, one file each

Module	Owns
`src/extension.ts`	activation, the output channel a build writes to, and the change signal
`src/projectHub.ts`	~~one project in depth~~ — removed: it read the same counts the Library and the dashboard already show, and clicking a project now selects it and opens its pages
`src/projectPage.ts`	the form: name, version, audience, owner, sources, outputs
`src/project.ts`	the commands that act on a project, and the argument tolerance they need to be called from a row
`src/removal.ts`	removing a project's folder with it: the question that names the folder, the delete, and what happens to the sources that recorded a path inside it
`src/projectBuild.ts`	building a project's outputs: the generator first when the repository has one, then one script per ticked output, judged by the files they left behind
`src/surface.ts`	the two view types
`src/store.ts` · `src/ui.ts` · `src/repositories.ts` · `src/aiModule.ts`	what this module reads of the store, the UI rules, the repository scripts and the AI — each in one file, because an extension cannot import another extension's sources

Commands

`temcoProjects.newProject` · `openProject` · `editProject` · `removeProject` · `selectProject` · `clearProject` · `addSource` · `buildProject` — eight, each doing one thing to a project.

Boundary rules

- **A project is a name and a set of relationships**, not a copy of anything. Its sources stay in their repositories, its pages stay where they are written, and the store holds the rows that tie them together.
- **The pages a project has come from the store.** A read recorded them; this module does not parse a registry, which is why it needs nothing from `temco-docs`.
- **A build is invoked, never reimplemented.** The script is the product repository's own; it runs in that repository's folder, its exit code is reported and never trusted, and the verdict is the file that did not exist before and does now. Every ticked output that has a script runs, once each — asking for a site and a PDF runs both. The deploying form of the help build is never invoked.

- **Removing a project takes its folder with it.** The row and everything recorded against it go, and so does the folder the project owns — `<projectsFolder>/<slug>/` (`temcoRepositories.projectsFolder`, or the working area when it is empty) — holding its `docs/`, its build and what that build produced. The confirmation names that folder before it is deleted. No origin is ever written to and nothing is ever pushed; a clone in the shared `source/` folder is untouched.
- **The AI is optional.** This module declares `temco-store`, `temco-ui` and `temco-repositories`, never `temco-ai`: its pages ask the AI for facts when that module is installed and say so when it is not.
- **The scripts vocabulary is the repositories module's.** Which of the repository's own scripts exist, and what each is for, is a fact about a repository — this module reads the list (`src/repositories.ts`) rather than keeping a second copy.